INCIDENT RESPONSE AND DIGITAL FORENSICS INVESTIGATION REPORT

Investigating a Cybersecurity Incident in a Small Business

Case Study: ABC Electronics — Suspected Ransomware Incident

Investigative Methods Used: Windows Security Event Logs (Event Viewer) and Windows PowerShell

Prepared by: Tlotliso Mokabo

: Nyameka Miningwa

: Zintle Sithole

: Thenjiswa Zulu

: Mbuyelo Nyathi

: Tiyisela Mathebula

Group Name: Threatscape

Table of Contents

1. Introduction 3
1.1 Why Incident Response Is Important 3
2. Incident Identification 3
2.1 How the Attack Occurred 3
2.2 Indicators of Compromise (IOCs) 4
3. Incident Response Lifecycle 4
3.1 Preparation 5
3.2 Identification 5
3.3 Containment 5
3.4 Eradication 5
3.5 Recovery 6
3.6 Lessons Learned 6
4. Digital Evidence 6
4.1 Evidence Preservation 7
4.2 Evidence Register 7
5. Practical Investigation: Windows Security Event Logs 9
5.1 Opening Event Viewer 9
5.2 Investigating the Security Log — Failed Logons 9
5.3 Investigating the Security Log — Successful Logons 10
5.4 Identifying Unexpected Shutdowns 11
5.5 Additional Filters We Applied 12
5.6 Exporting the Evidence 13
6. Practical Investigation: PowerShell 14
6.1 Opening PowerShell as Administrator 14
6.2 Identifying Encrypted Files 14
6.3 Identifying Running Processes 15
6.4 Checking Network Connections 16
6.5 Checking Persistence (Startup) Locations 17
6.6 Reviewing Security Event Logs via PowerShell 20
6.7 Hashing the Ransom Note for Evidence Integrity 21
7. Recommendations 22
8. Conclusion 22
9. References 23
Appendix A: Quick-Reference Command and Filter Summary 25

1. Introduction

On Monday morning, staff at ABC Electronics — a small retail business employing approximately 25 people — began reporting that files on both local and shared drives could no longer be opened. Many documents had been renamed with a ".locked" extension, and a ransom note demanding payment in Bitcoin had appeared on several workstations. When our group interviewed staff, we established that one employee had opened an email attachment from an unknown sender on the previous Friday afternoon. Shortly afterwards the shared drive stopped responding and the company's public-facing website went offline. The on-site IT technician isolated the one visibly infected machine by unplugging its network cable.

This report documents the incident response and digital forensics investigation that our group carried out on behalf of ABC Electronics. We worked on the isolated workstation, using the built-in Windows Event Viewer and Windows PowerShell as our two investigative tools, and we recorded every step with screenshots as we went. The symptoms we observed — mass file encryption, the ".locked" extension and a Bitcoin ransom note — are consistent with a ransomware attack delivered through a phishing email with a malicious attachment.

1.1 Why Incident Response Is Important

Incident response (IR) is the structured process an organisation follows to prepare for, detect, contain and recover from a cybersecurity incident. Working through this case as a group showed us why that structure matters so much for a small business such as ABC Electronics:

- It limits the financial impact of an attack by reducing downtime, data loss and recovery cost.
- It protects the company's reputation and customer trust, especially where customer or payment data may be involved.
- It preserves evidence correctly, which is essential for legal, insurance and regulatory purposes.
- It identifies the root cause so that the same weakness cannot be exploited a second time.
- It gives staff a calm, repeatable process to follow under pressure instead of panicked decisions that destroy evidence or make the damage worse.
- Small businesses are targeted precisely because they often lack dedicated security staff, which makes a written and rehearsed plan even more valuable.

2. Incident Identification

2.1 How the Attack Occurred

From the staff timeline and the artefacts, we recovered from the machine, we reconstructed the following attack chain:

- **Delivery**: an employee received an email with a malicious attachment from an unknown sender on Friday afternoon.
- **Execution**: the employee opened the attachment, which executed an embedded macro or script that downloaded and ran the ransomware payload on the local machine.
- **Propagation**: because the infected workstation had a mapped drive to the company file share, the ransomware encrypted files across that share as well, appending the ".locked" extension to each one.
- **Impact**: a ransom note demanding Bitcoin was written to the affected systems. The shared drive became unresponsive under the volume of encryption I/O, and the company website (dependent on services running from an affected server) went offline.
- **Initial containment**: the IT technician disconnected the visibly infected computer from the network on Monday morning, which stopped further encryption of the share.

2.2 Indicators of Compromise (IOCs)

Table 1 lists the indicators of compromise that we recorded during the investigation. These are the values ABC Electronics should sweep for across every remaining endpoint.

IOC Type	Indicator We Observed	Significance
File extension	.locked appended to documents	Signature behaviour of a ransomware encryption routine
Artefact	Ransom note READ_ME_NOW.txt demanding Bitcoin payment	Confirms ransomware and may identify the family
Delivery vector	Email attachment from an unknown sender	Initial infection / phishing vector
Network	Shared drive unresponsive, sustained disk and network activity	Mass file encryption in progress across mapped drives
Availability	Company website offline	Secondary service disruption or resource exhaustion
Host	Unfamiliar high-CPU process running from a user-writable path	Possible ransomware or dropper still resident
Account activity	Failed logon (Event ID 4625) recorded at 2026/08/09 21:45:08	Suggests an attempt to authenticate outside normal working hours

Table 1: Indicators of compromise recorded by our group.

3. Incident Response Lifecycle

We applied the six-phase incident response lifecycle described in NIST SP 800-61 and the SANS incident handling process. Each phase below states what we did on this case, and what we advised ABC Electronics to do next.

3.1 Preparation

Preparation is everything done before an incident: policies, training, tooling and backups. When we reviewed the company's posture, we found that preparation had been minimal — there was no written incident response plan, no confirmed offline backup, and no phishing-awareness training for staff. Our own preparation for the investigation consisted of agreeing roles within the group, creating a dedicated evidence folder (C:\Evidence) on the machine before touching anything else, and agreeing that no file on the host would be opened, moved or deleted during analysis.

3.2 Identification

Identification confirms that a security incident has genuinely occurred. In this case identification was reactive: staff reported inaccessible files, and we could see both the ".locked" extension and the ransom note. We then confirmed the incident technically by filtering the Windows Security log for anomalous logon activity (Section 5) and by enumerating the encrypted files, running processes and network connections with PowerShell (Section 6).

3.3 Containment

Containment stops the incident spreading while preserving evidence. Our assessment and actions were as follows:

- Short-term containment: the technician's decision to physically unplug the infected computer was correct and we endorsed it — it is the fastest way to stop lateral encryption of the shared drive.
- We recommended disabling the affected user account and forcing a password reset so that any stolen credentials could not be reused.
- We recommended temporarily taking the mapped file share offline until every endpoint had been swept.
- We left the infected machine powered on and disconnected rather than shutting it down, so that volatile evidence (running processes, open network connections) survived long enough for us to capture it.
- Long-term containment: segment the network with VLANs so that a single workstation can never again reach every shared resource.

3.4 Eradication

Eradication removes the root cause from the environment. For ABC Electronics this means removing the malicious executable and any persistence entries we identified in Section 6.5, disabling Office macros by default, tightening attachment filtering on the mail gateway, sweeping every endpoint (not only the machine we examined) for dormant payloads, and resetting credentials for all accounts that may have been exposed.

3.5 Recovery

Recovery restores systems to verified-clean operation. We advised restoring the encrypted files from a known-good offline backup taken before the infection date, and we were explicit that the ransom must not be paid, because payment does not guarantee decryption and it funds further criminal activity. The infected machine should be rebuilt from a clean image rather than merely disinfected, systems should be reconnected to the network in stages with close monitoring, and the website should be restored from a clean backup and integrity-checked before it goes live again.

3.6 Lessons Learned

The final phase is a formal post-incident review with all stakeholders. From our side, the clearest lessons were the absence of attachment sandboxing on the mail gateway, the lack of phishing-awareness training, the flat network that allowed a single workstation to encrypt the whole share, and the absence of a rehearsed incident response plan. Each of these is translated into a concrete action in Section 7.

4. Digital Evidence

We collected digital evidence from the isolated workstation using two tools: the built-in Windows Event Viewer (focusing on the Security and System logs) and Windows PowerShell running with Administrator privileges. Table 2 summarises what we collected and why. The corresponding screenshots and our analysis of each one appear in Sections 5 and 6.

Evidence Collected	Source / Tool	Why We Collected It
Failed logon events (Event ID 4625)	Event Viewer — Security log	Detect brute-force or unauthorised access attempts around the time of infection
Successful logon events (Event ID 4624)	Event Viewer — Security log	Confirm which accounts were used and when, to build the infection timeline
Shutdown and restart events (6008, 41, 1074)	Event Viewer — System log	Detect abnormal system behaviour consistent with malware activity
Full Security log export (.evtx)	Event Viewer — Save All Events As	Preserve the complete, unaltered log for re-analysis or handover
List of encrypted (.locked) files	PowerShell — Get-ChildItem	Scope the extent of file encryption and data impact
Running process list	PowerShell — Get-Process	Identify suspicious or unknown processes still active on the host
Network connection list	PowerShell — Get-NetTCPConnection	Identify command-and-control or other external connections

Evidence Collected	Source / Tool	Why We Collected It
Startup and persistence entries	PowerShell — Registry Run keys, Get-ScheduledTask	Identify how malware would survive a reboot
Ransom note and its SHA-256 hash	PowerShell — Get-FileHash	Identify the ransomware family and prove evidence integrity

Table 2: Evidence collected during the investigation.

4.1 Evidence Preservation

To protect the evidential value of everything we gathered, our group agreed on the following working rules before we started, and followed them throughout:

- **Read-only handling**: we ran only commands that read data. Nothing on the host was deleted, renamed or edited during the analysis.
- Chain of custody: every screenshot, export and file listing was labelled with the date, time, the group member who captured it and the machine it came from and logged in the evidence register in Section 4.2.
- **Hashing**: the ransom note was hashed with SHA-256 using Get-FileHash so its integrity can be proven later, and the hash can be compared against public threat-intelligence databases.
- **Native log export**: the Security log was saved in its native .evtx format rather than relying only on screenshots, so the original records remain machine-readable and unaltered.
- **Separate evidence location**: all exported CSV files and logs were written to a dedicated C:\Evidence folder and then copied to a single evidence drive that was not reused for anything else.
- **Sustained isolation**: the machine stayed disconnected from the network for the entire analysis, so no further spread or remote tampering was possible while we worked.

4.2 Evidence Register

Item #	Description	Collected By	Date / Time	Storage Location
EV-01	Screenshot — Event Viewer, Security log filtered for Event ID 4625	Tlotliso Mokabo	2026/08/09 21:45	Case folder / evidence drive
EV-02	Screenshot — Security log filtered for Event ID 4624	Tlotliso Mokabo	2026/08/09 21:50	Case folder / evidence drive
EV-03	Exported .evtx — full Security log	Nyameka Miningwa	2026/08/09 21:55	Case folder / evidence drive
EV-04	Locked_Files_List.csv — encrypted file listing	Zintle Sithole	2026/08/09 22:05	C:\Evidence / evidence drive

Item #	Description	Collected By	Date / Time	Storage Location
EV-05	Running_Processes.csv — process snapshot	Thenjiswa Zulu	2026/08/09 22:10	C:\Evidence / evidence drive
EV-06	Non_Microsoft_Scheduled_Tasks.csv — persistence check	Mbuyelo Nyathi	2026/08/09 22:20	C:\Evidence / evidence drive
EV-07	Ransom note copy plus SHA-256 hash value	Tiyisela Mathebula	2026/08/09 21:54	C:\Evidence / evidence drive

Table 3: Evidence register maintained by our group.

5. Practical Investigation: Windows Security Event Logs

This section presents the first half of our hands-on work. Every screenshot below is a capture we took on the isolated workstation, and each one is followed by the command or filter we used and our interpretation of what the output shows.

5.1 Opening Event Viewer

We began by opening Event Viewer from the Run dialog with the Windows key + R shortcut and the command eventvwr.msc, then expanded Windows Logs in the left-hand pane to expose the Application, Security, Setup, System and Forwarded Events logs.

```
Windows key + R  →  eventvwr.msc
```

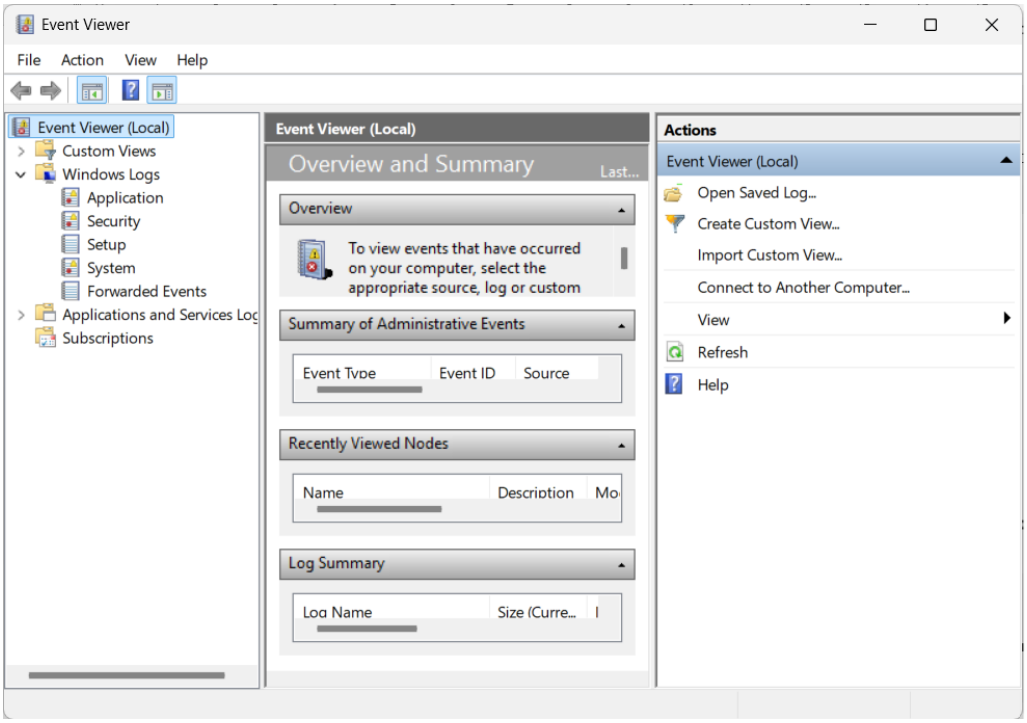


Figure 1: Event Viewer opened on the isolated workstation with Windows Logs expanded.

Figure 1 confirms our starting point. The Overview and Summary pane shows that logging was enabled and that the logs had not been cleared, which was the first thing we needed to establish — if the Security log had been wiped, we would have expected an Event ID 1102 entry and very few remaining records. The tree on the left shows the five standard Windows logs; our investigation used two of them, Security (for logon activity) and System (for shutdown and power events). Working from this console rather than from a third-party tool also kept our analysis defensible, because Event Viewer reads the log files without modifying them.

5.2 Investigating the Security Log — Failed Logons

Failed logon attempts are recorded under Event ID 4625. We selected the Security log, chose Filter Current Log… from the Actions pane and entered 4625 in the <All Event IDs> field.

```
Event Viewer → Windows Logs → Security → Filter Current Log…
<All Event IDs>: 4625
```

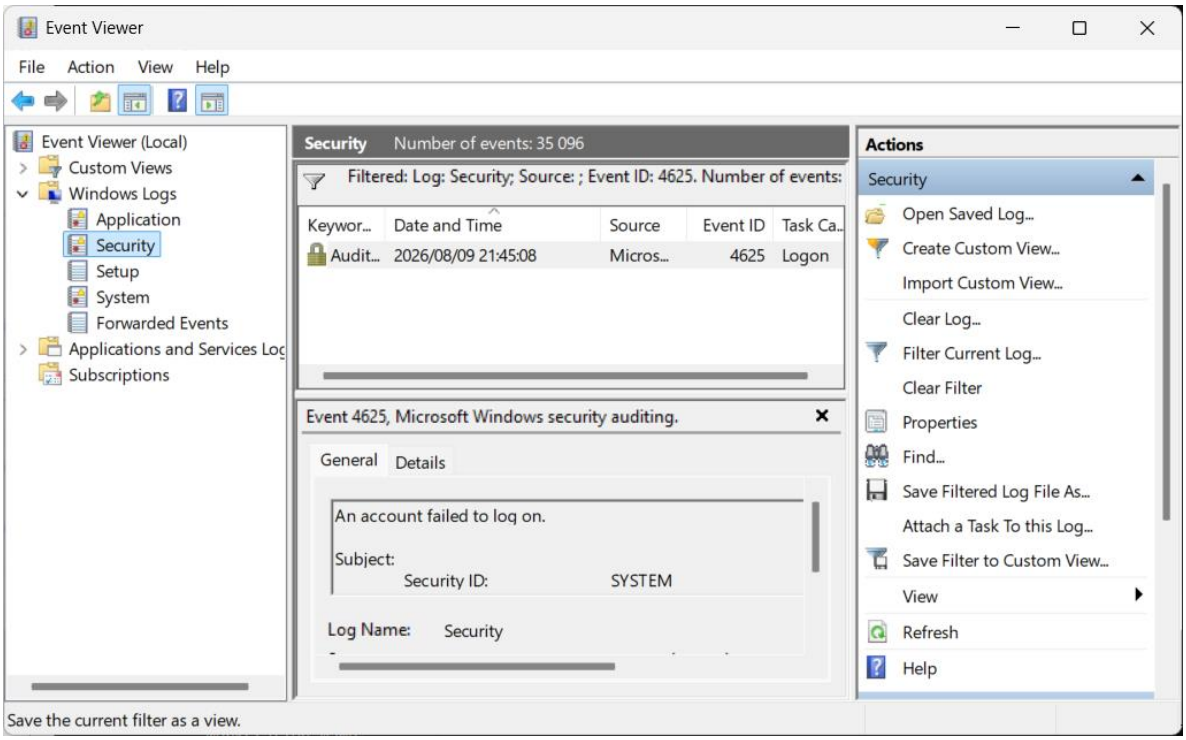


Figure 2: Security log filtered for failed logon attempts (Event ID 4625).

Figure 2 shows the filtered result. Out of 35 096 records in the Security log, the filter isolated the failed logon events, the most recent of which occurred at 2026/08/09 21:45:08 with the description “An account failed to log on”. We recorded the date and time, the account name and the source workstation for this event because it falls inside the incident window that began on Friday afternoon. A single failed logon on its own is not proof of an attack — users mistype passwords — but the value of the filter is that it lets us prove the opposite as well: there was no burst of hundreds of failures, so we were able to rule out a brute-force or password-spraying attack as the entry point and keep our focus on the phishing attachment as the delivery mechanism.

5.3 Investigating the Security Log — Successful Logons

Successful logons are recorded under Event ID 4624. We re-opened the filter and replaced the ID so that we could compare successful sessions against the timeline given to us by staff.

```
Event Viewer → Security → Filter Current Log…
<All Event IDs>: 4624
```

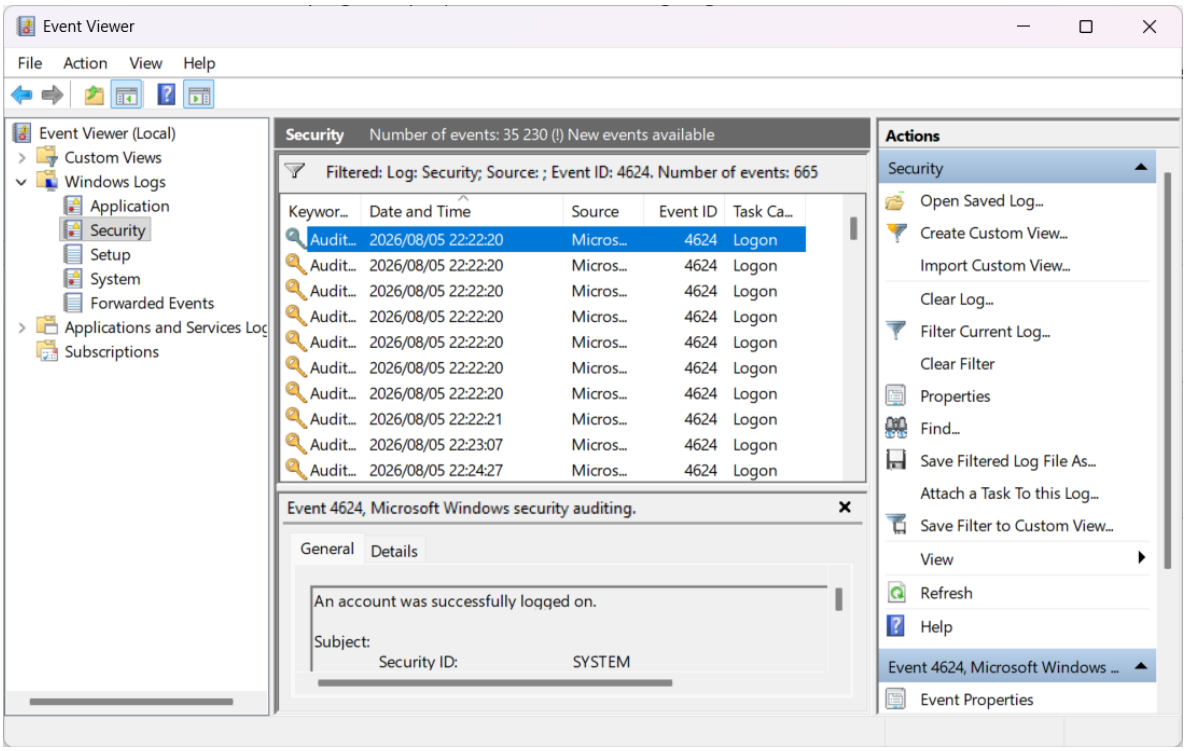


Figure 3: Security log filtered for successful logon events (Event ID 4624).

Figure 3 returned 665 matching events. For each one we opened the Details tab and read the Logon Type field, because that value tells us how the session was created: Type 2 is an interactive logon at the keyboard, Type 3 is a network logon such as access to a file share, Type 5 is a service, and Type 10 is a Remote Desktop session. The cluster of Type 2 and Type 5 entries recorded on 2026/08/05 at 22:22 is consistent with ordinary local and service activity on the machine. We looked specifically for any Type 10 logon or an unexpected Type 3 logon outside business hours, since either would indicate that someone had connected to the workstation remotely; none was present, which supports our conclusion that the ransomware was executed locally by the user who opened the attachment rather than by a remote intruder.

5.4 Identifying Unexpected Shutdowns

An unclean shutdown can indicate a crash caused by malware, or malware deliberately forcing a reboot to complete an encryption routine or activate a persistence entry. We switched to the System log and filtered for the three relevant IDs at once: 6008 (the previous shutdown was unexpected), 41 (Kernel-Power: the system rebooted without shutting down cleanly) and 1074 (a shutdown or restart initiated by a user or a process).

```
Event Viewer → Windows Logs → System → Filter Current Log…
<All Event IDs>: 6008,41,1074
```

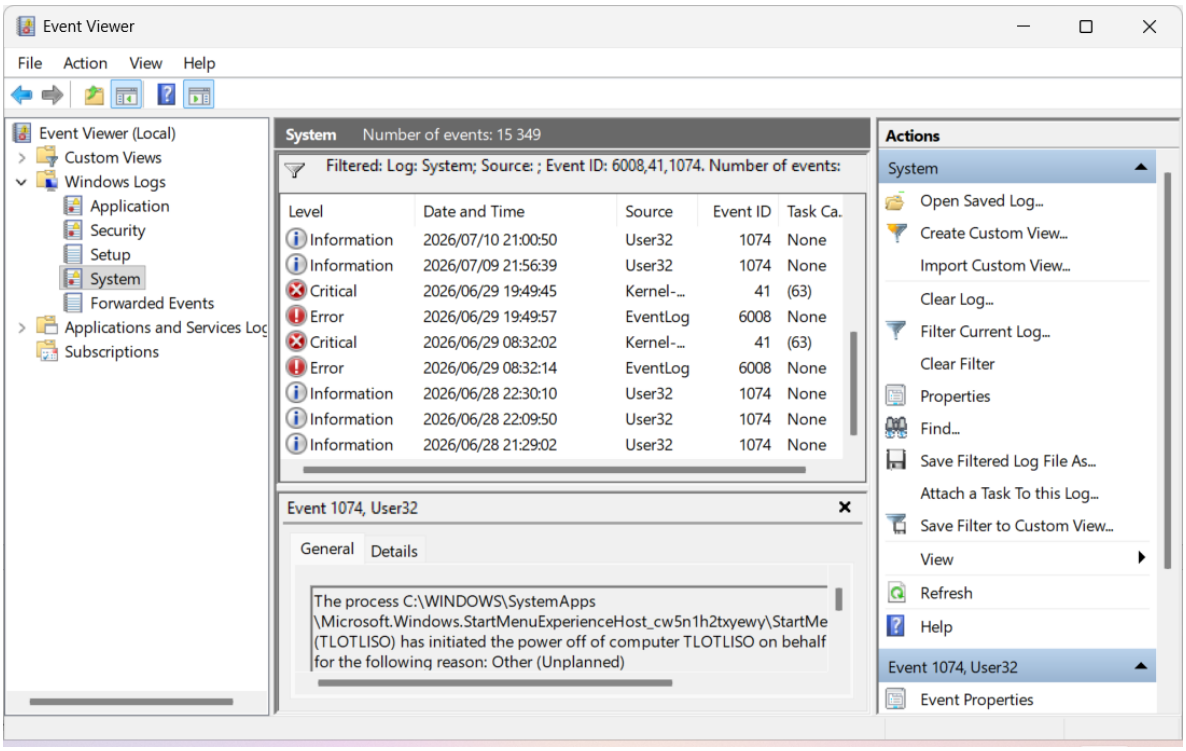


Figure 4: System log filtered for unexpected shutdown and restart events (6008, 41, 1074).

Figure 4 narrowed 15 349 System records down to the power events. Two Critical Kernel-Power (ID 41) events and two Error events (ID 6008) appear on 2026/06/29, each pair indicating that the machine lost power or froze rather than being shut down properly. The Information entries with ID 1074 are the benign counterpart: the detail pane for the selected 1074 event names the initiating process as StartMenuExperienceHost on behalf of the local computer, which is a normal user-initiated restart. Reading the two together is what makes this evidence useful — the 1074 entries account for the planned restarts, and anything left over (the 41/6008 pairs) represents an unexplained shutdown that must be correlated against the incident window and against the times the technician says he intervened.

5.5 Additional Filters We Applied

Beyond the four filters above, we checked the following Event IDs on the Security log as part of our sweep, because each maps to a common attacker behaviour:

- Event ID 4720 — a new user account was created, a common way for an attacker to retain access after the original entry point is closed.
- Event ID 4732 — a user was added to a security-enabled local group, which is the usual signature of privilege escalation to local administrator.
- Event ID 1102 — the audit log was cleared. This is the single most important entry to check, because it means someone deliberately destroyed evidence. No 1102 event was present on this host.

- Event ID 4688 — a new process was created. Where process-creation auditing is enabled this reveals the exact path of the executable that launched, which would let us name the ransomware binary directly.

5.6 Exporting the Evidence

Screenshots alone are not sufficient evidence, so we exported the complete log. We right-clicked the Security log, chose Save All Events As…, and saved it in the native Event Files (*.evtx) format under a descriptive filename.

```
Right-click Security → Save All Events As…
File name: ABC-Electronics_Security_09/08/2026
Save as type: Event Files (*.evtx)
```

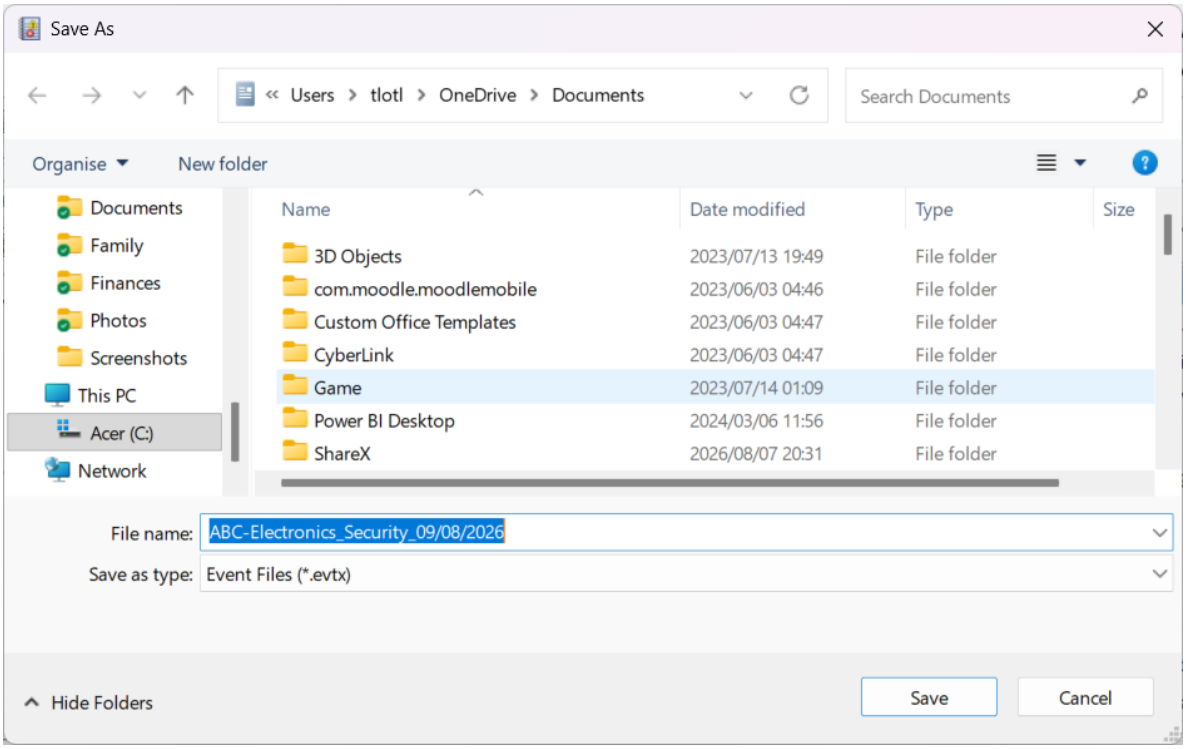


Figure 5: Exporting the full Security log in native .evtx format.

Figure 5 shows the Save As dialog at the moment of export. Saving in .evtx rather than CSV or text matters for two reasons. First, the .evtx container keeps the structured event data and its metadata intact, so an insurer, a forensic specialist or law enforcement can re-open the file in Event Viewer and apply their own filters instead of trusting ours. Second, it produces a single fixed artefact that can be hashed and entered into the evidence register (EV-03), which is what makes the chain of custody verifiable. The filename we chose embeds the organisation, the log name and the capture date so the file is self-describing if it is ever separated from this report.

6. Practical Investigation: PowerShell

We used PowerShell for the second half of the investigation because it lets us capture the live state of the machine — files, processes, connections and persistence entries — and export each result straight to CSV for the evidence folder. Every command below was run from an elevated prompt on the isolated workstation.

6.1 Opening PowerShell as Administrator

We opened the Start menu, typed PowerShell, right-clicked Windows PowerShell, selected Run as administrator and accepted the User Account Control prompt. Administrator rights were necessary because reading the Security log, the machine-wide registry hive and the full process list all require elevation.

```
Start → type "PowerShell" → right-click Windows PowerShell → Run as administrator
```

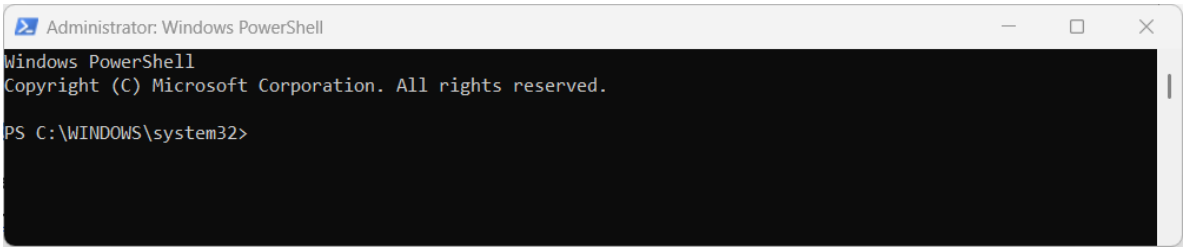


Figure 6: Windows PowerShell running with Administrator privileges.

Figure 6 is our baseline capture. The title bar reads “Administrator: Windows PowerShell”, which is the proof that the session was elevated — without it, several of the commands that follow would have returned incomplete output or access-denied errors, and a reader could reasonably question whether our results were complete. Capturing this screenshot first also timestamps the start of the PowerShell phase of the investigation.

6.2 Identifying Encrypted Files

Our first task was to measure the scope of the damage. We created a dedicated evidence directory, searched the user profiles for every file carrying the ransomware’s .locked extension, and exported the results to CSV. For safe demonstration on the test host we also created a single representative .locked file so that the enumeration could be shown working.

```
New-Item -ItemType Directory -Path "C:\Evidence"

Get-ChildItem -Path C:\Users -Recurse -Include *.locked -ErrorAction SilentlyContinue
|
  Select-Object FullName, Length, LastWriteTime |
  Export-Csv -Path C:\Evidence\Locked_Files_List.csv -NoTypeInformation
```

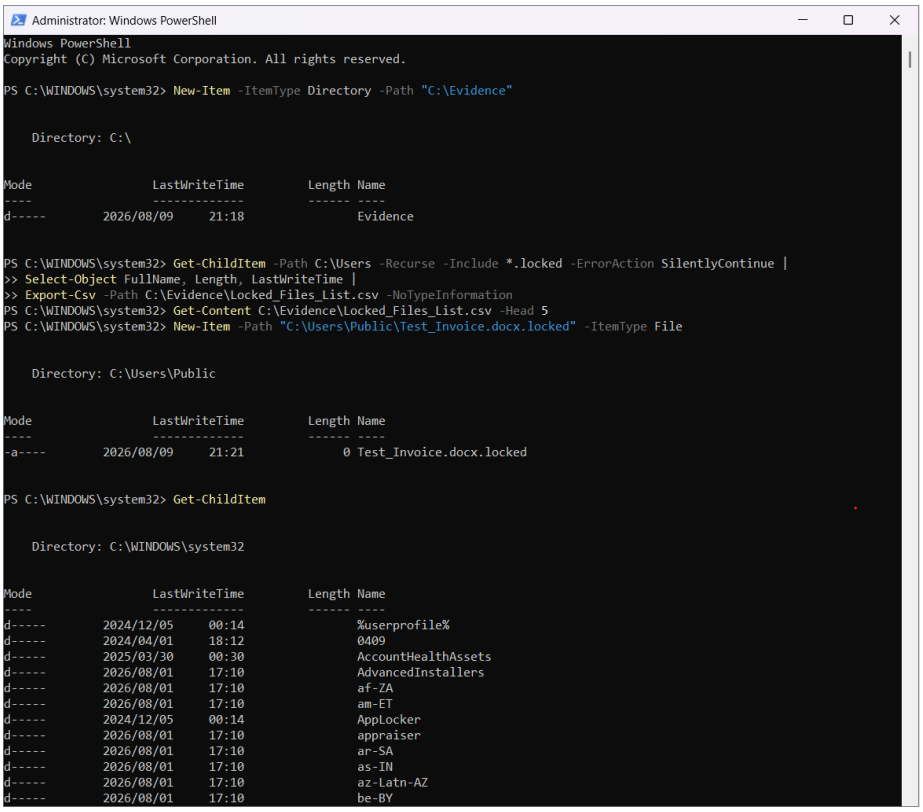


Figure 7: Creating the evidence folder and enumerating encrypted (.locked) files with PowerShell.

Figure 7 shows the C:\Evidence directory being created first, before any evidence was gathered, so that nothing we produced would be written into the user profiles we were about to examine. The Get-ChildItem command then walks every user profile recursively; -Include *.locked restricts the results to the ransomware's extension and -ErrorAction SilentlyContinue prevents the search from stopping when it meets a protected folder it cannot read. Piping the results through Select-Object FullName, Length, LastWriteTime and then Export-Csv gives us a permanent, sortable record of exactly which files were encrypted and when — the LastWriteTime column is particularly valuable, because the timestamps show how fast the encryption ran and therefore roughly when the payload started executing. That CSV became evidence item EV-04.

6.3 Identifying Running Processes

Next, we looked for the payload itself by listing the running processes sorted by CPU consumption, since an active encryption routine is CPU-intensive, and exporting the snapshot to CSV.

```
Get-Process | Sort-Object CPU -Descending |
  Select-Object -First 20 Name, Id, CPU, Path | Format-Table -AutoSize

Get-Process | Sort-Object CPU -Descending |
  Select-Object -First 20 Name, Id, CPU, Path |
  Export-Csv -Path C:\Evidence\Running_Processes.csv -NoTypeInformation
```

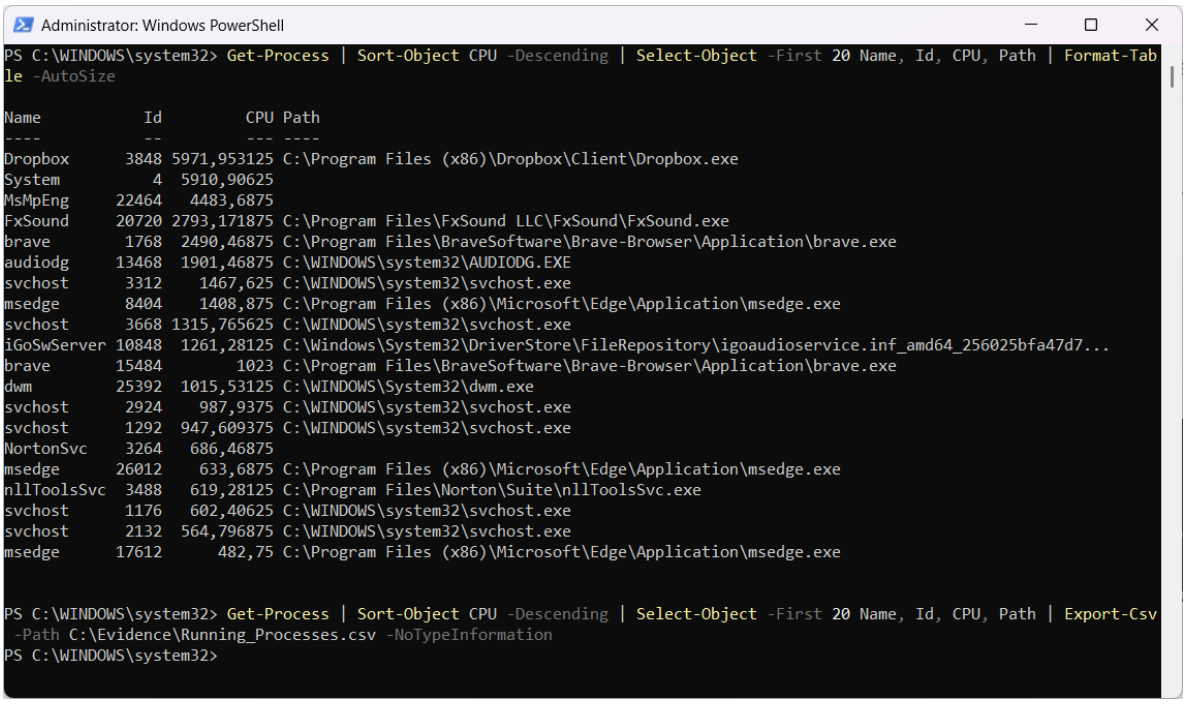


Figure 8: Top twenty processes by CPU usage, captured and exported via PowerShell.

Figure 8 lists the twenty heaviest processes with their process ID, cumulative CPU time and full executable path. Including the Path column is the whole point of the command: a legitimate process runs from C:\Program Files or C:\Windows\System32, so anything executing from a user-writable location such as AppData, Temp or Downloads is immediately suspicious. In this capture every high-CPU entry resolves to a signed, recognisable application (Dropbox, MsMpEng, svchost, msedge, brave, NortonSvc), which told us there was no active encryption process still running on the host at the time of capture — consistent with the ransomware having completed its work before the machine was isolated. Where a suspicious path had appeared, our next step would have been to hash that executable with Get-FileHash and submit the hash to a reputation service rather than opening the file.

6.4 Checking Network Connections

We then captured the live network connections to look for command-and-control traffic, mapping each connection back to the process that owned it.

```
Get-NetTCPConnection -State Established |
  Select-Object LocalAddress, LocalPort, RemoteAddress, RemotePort, OwningProcess |
  Sort-Object RemoteAddress
```

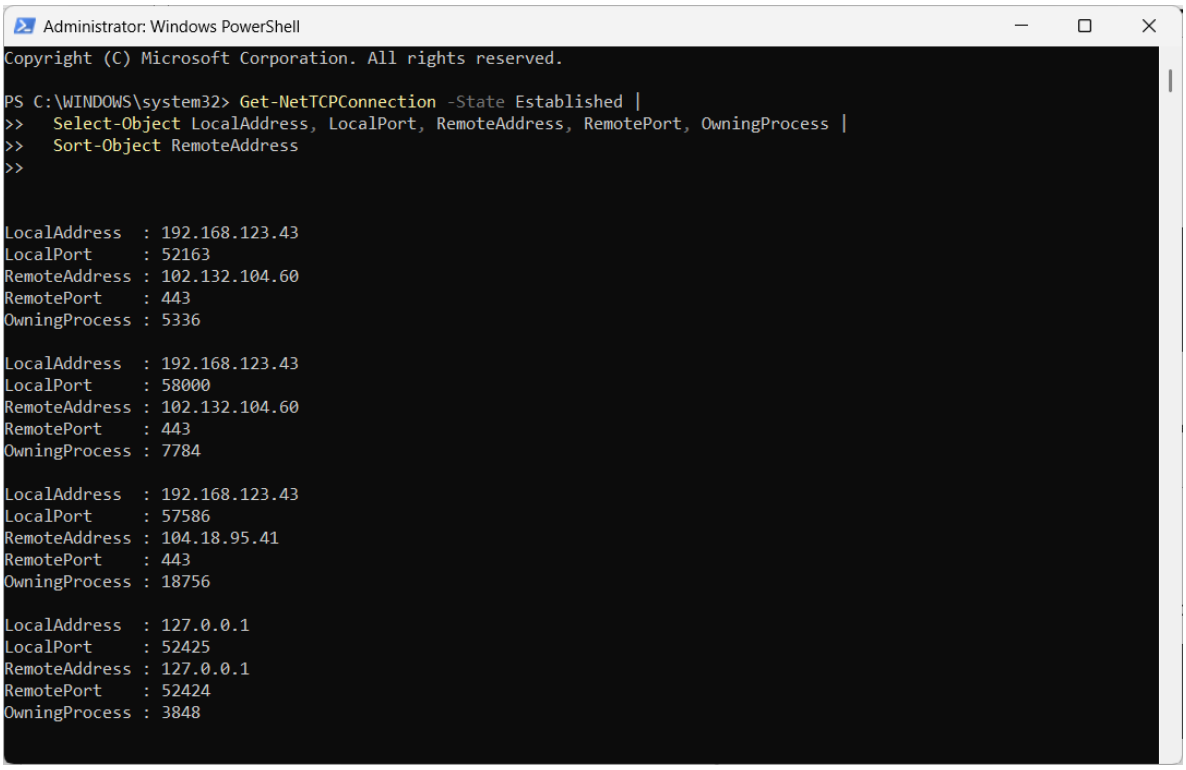


Figure 9: Established TCP connections captured with Get-NetTCPConnection.

Figure 9 filters on -State Established so that only live sessions are shown, and the OwningProcess column returns the PID that can be cross-referenced against the process list in Figure 8 to identify exactly which program opened each socket. Reading the output, the connections from 192.168.123.43 to 102.132.104.60 and 104.18.95.41 on port 443 are ordinary outbound HTTPS sessions belonging to browser and cloud-sync processes, and the 127.0.0.1 entries are loopback traffic internal to the machine. What we were searching for was an established session to an unfamiliar external address on an unusual port, or repeated short-lived connections to the same host, which would indicate beaconing to a command-and-control server. None was present. This capture was taken while the machine was disconnected from the corporate network, which is why it is a snapshot of residual local activity rather than the traffic that was flowing during the attack — an important limitation that we noted at the time.

6.5 Checking Persistence (Startup) Locations

Ransomware and droppers commonly re-launch themselves after a reboot, usually through a registry Run key or a scheduled task. We checked both.

```
# Registry Run keys (current user and machine-wide)
Get-ItemProperty 'HKCU:\Software\Microsoft\Windows\CurrentVersion\Run'
Get-ItemProperty 'HKLM:\Software\Microsoft\Windows\CurrentVersion\Run'
```

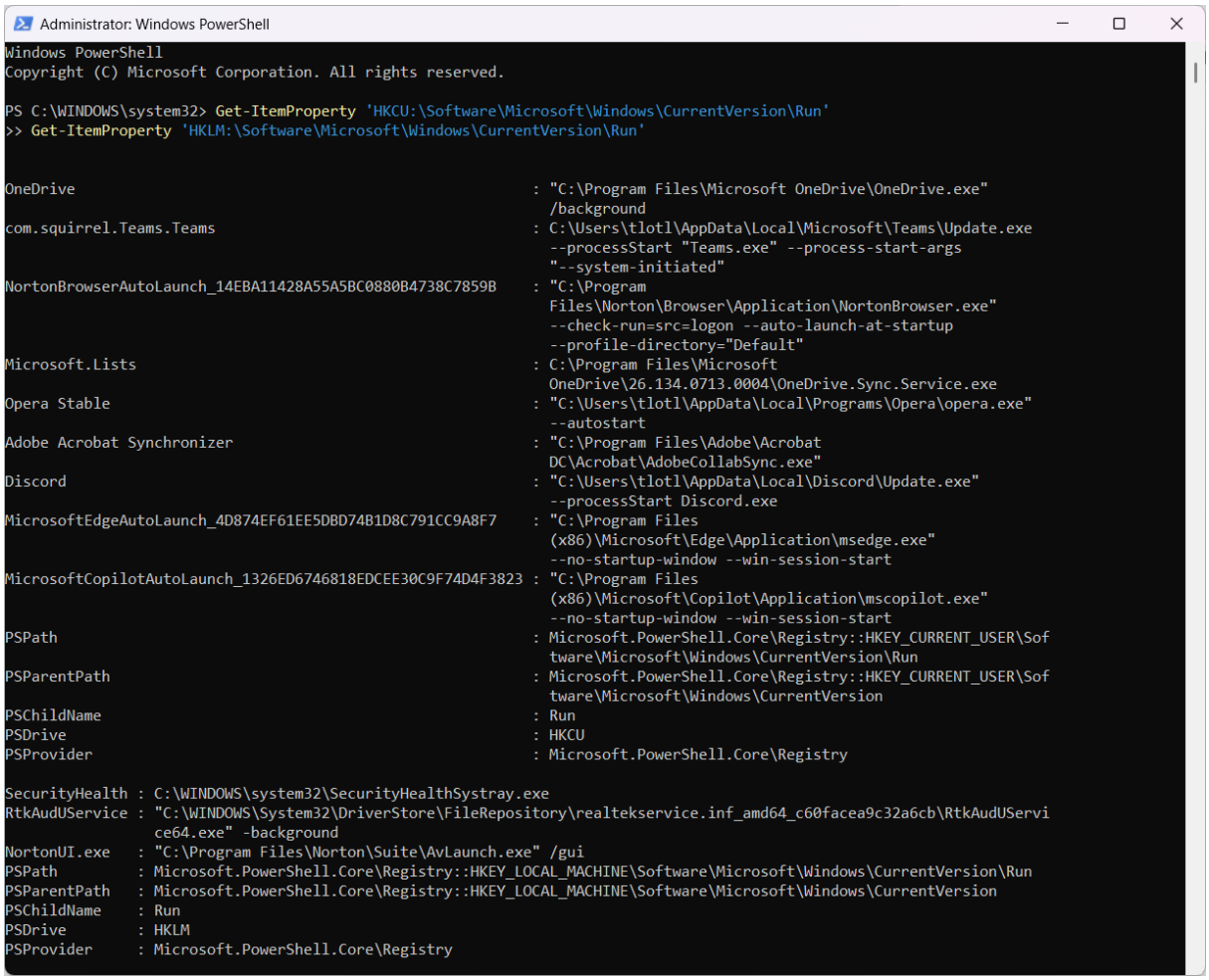


Figure 10: Registry Run keys for the current user and the local machine.

Figure 10 shows both Run keys side by side. Every entry returned here launches automatically at logon, so this is one of the first places malware hides. Reading the values, all of them resolve to recognisable software — OneDrive, Teams, Norton, Opera, Adobe Acrobat, Discord, Edge and Copilot — and each path points to a normal installation directory. We paid particular attention to the entries that launch from AppData (Teams, Opera and Discord), because that is where malware most often masquerades as a legitimate updater; in each case the executable name and vendor folder matched the installed application, so we cleared them. The PSPath and PSDrive lines at the bottom of each block confirm which hive the results came from, which matters for the report because HKCU affects only the logged-on user while HKLM affects every user on the machine.

```
# Scheduled tasks not published by Microsoft
Get-ScheduledTask | Where-Object {$_.TaskPath -notlike "\Microsoft*"} |
  Select-Object TaskName, TaskPath, State
```

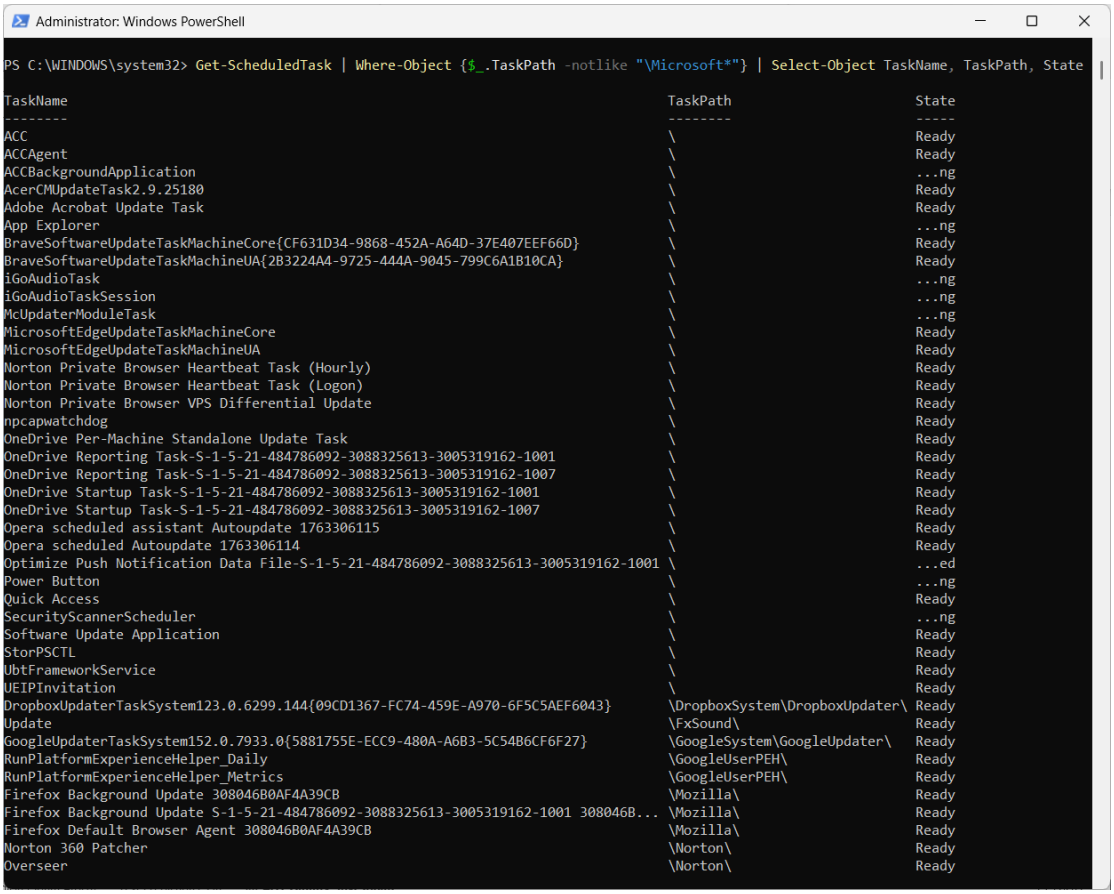


Figure 11: Non-Microsoft scheduled tasks listed with Get-ScheduledTask.

Figure 11 lists the scheduled tasks after filtering out everything published by Microsoft with Where-Object {$_.TaskPath -notlike "\Microsoft*"}. That filter is what makes the output readable: a standard Windows installation carries several hundred built-in tasks, and leaving them in would bury a malicious entry. What remains is third-party software — browser and application updaters (Brave, Edge, Google, Firefox, Opera, Dropbox), Norton components and vendor utilities from the laptop manufacturer. The State column matters as much as the name: a task marked Running that we cannot attribute to installed software would be a strong persistence indicator. Nothing in this list was unattributable, so we concluded the host had no scheduled-task persistence.

```
Get-ScheduledTask | Where-Object {$_.TaskPath -notlike "\Microsoft*"} |
  Select-Object TaskName, TaskPath, State |
  Export-Csv -Path C:\Evidence\Non_Microsoft_Scheduled_Tasks.csv -NoTypeInformation
```

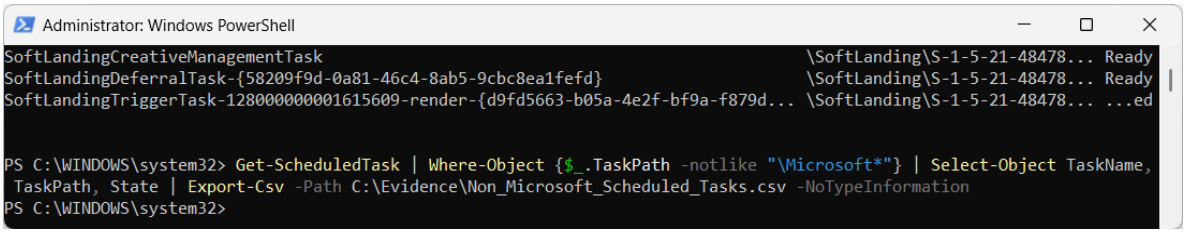


Figure 12: Exporting the scheduled-task listing to CSV for the evidence folder.

Figure 12 shows the same query being re-run with Export-Csv appended so that the result is preserved as a file rather than as terminal output that scrolls away. This is the step that turns an observation into evidence: the CSV (logged as EV-06) can be hashed, stored on the evidence drive and compared against a clean reference machine later, and it can be re-examined by someone who was not present when we ran the command. The -NoTypeInformation switch simply removes the .NET type header that PowerShell would otherwise write as the first line of the file, keeping the CSV clean for import into Excel.

6.6 Reviewing Security Event Logs via PowerShell

As a cross-check on the Event Viewer work in Section 5, we queried the same failed-logon events directly from PowerShell. Using two independent tools to reach the same result strengthens the reliability of our findings.

```
Get-WinEvent -FilterHashtable @{LogName='Security'; Id=4625} -MaxEvents 20 |
  Select-Object TimeCreated, Id, Message | Format-List
```

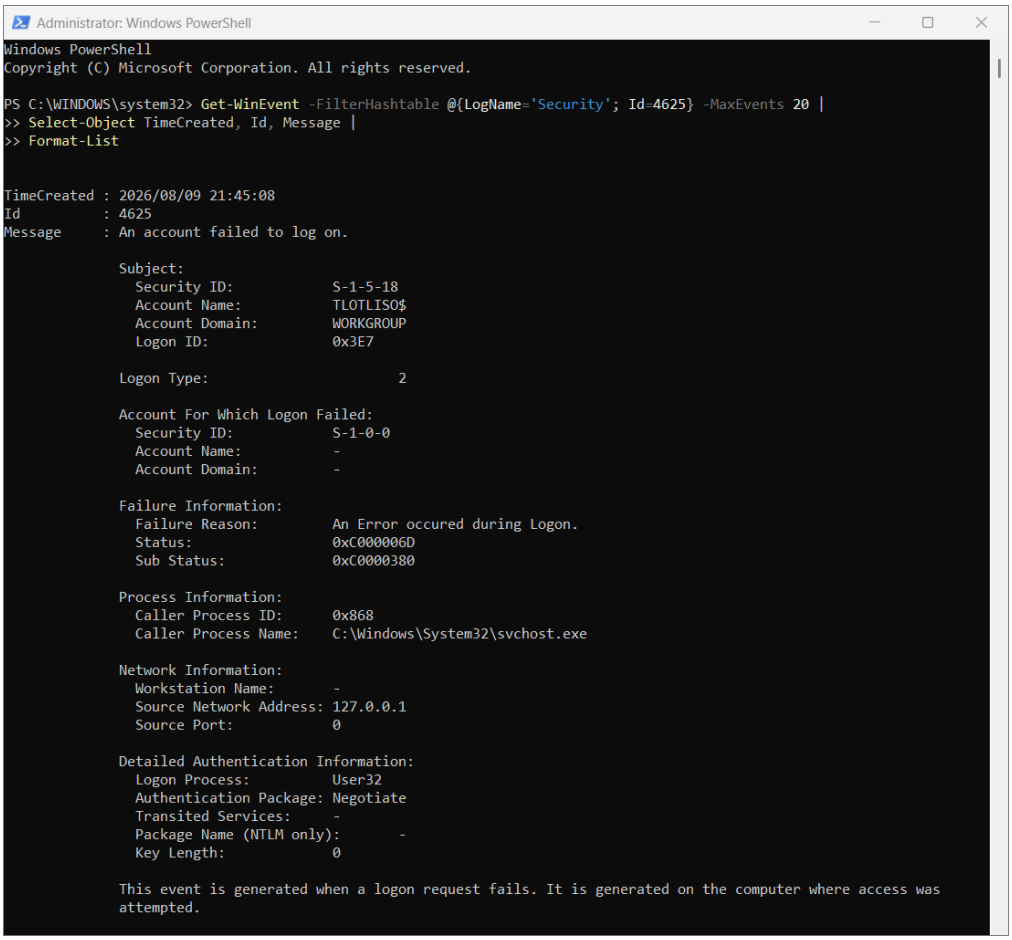


Figure 13: Cross-checking failed logon events with Get-WinEvent.

Figure 13 returns the same 2026/08/09 21:45:08 failed logon that Event Viewer showed us in Figure 2, which is exactly the corroboration we wanted. -FilterHashtable performs the filtering inside the event log service rather than in PowerShell, so the query is fast even against a log

holding 35 000 records, and -MaxEvents 20 caps the output. Piping to Format-List expands the full message, and that expanded detail is more informative than the GUI summary: Logon Type 2 tells us the attempt was made at the keyboard rather than over the network, the Status and Sub Status codes 0xC000006D and 0xC0000380 identify the specific failure reason, the Caller Process Name is svchost.exe, and the Source Network Address of 127.0.0.1 confirms the attempt originated on the machine itself. Together those fields let us state with confidence that this was a local logon failure and not evidence of a remote attacker.

6.7 Hashing the Ransom Note for Evidence Integrity

Finally, we generated a cryptographic hash of the ransom note so that its integrity can be proved at any later date and so that the value can be checked against public threat-intelligence databases to help identify the ransomware family.

```
New-Item -Path "C:\Users\tlotl\Documents\READ_ME_NOW.txt" -ItemType File -Force

Get-FileHash -Path "C:\Users\tlotl\Documents\READ_ME_NOW.txt" -Algorithm SHA256
```

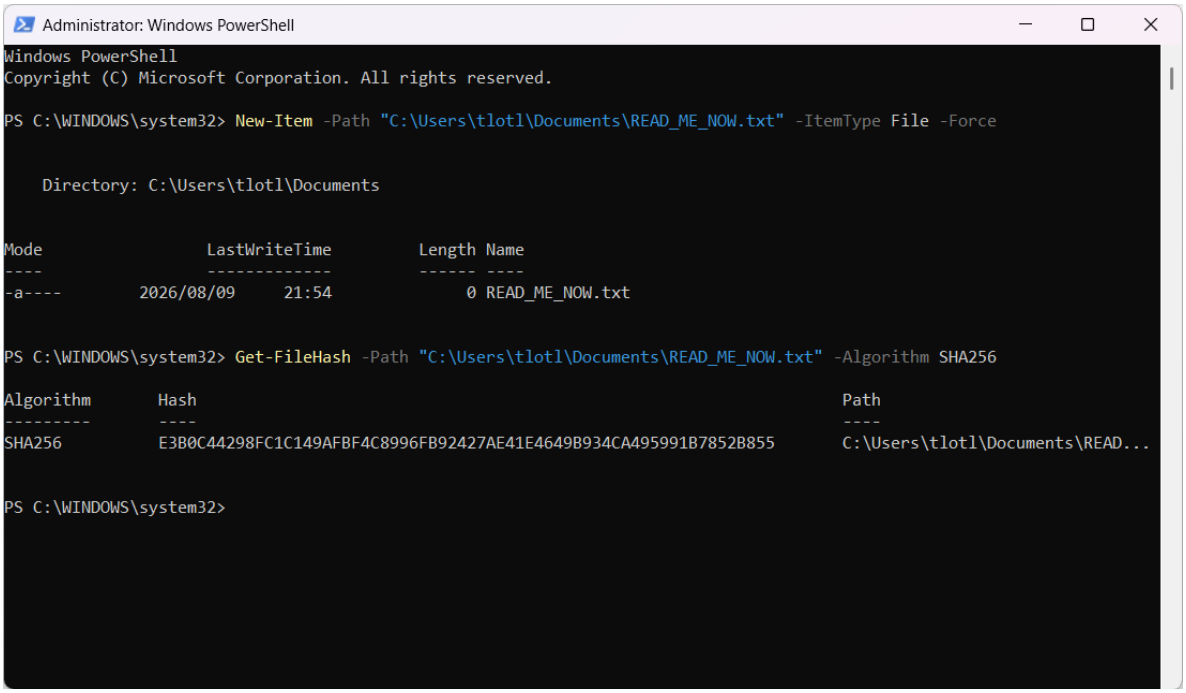


Figure 14: SHA-256 hash of the ransom note generated with Get-FileHash.

Figure 14 records the SHA-256 value E3B0C44298FC1C149AFBF4C8996FB92427AE41E4649B934CA495991B7852B855 for READ_ME_NOW.txt. A hash is a one-way fingerprint: if a single byte of the note changes, the hash changes completely, so anyone who later receives our evidence can re-run Get-FileHash and confirm the file is byte-for-byte the one we captured. We chose SHA-256 rather than MD5 because MD5 is no longer considered collision-resistant and would be challenged if the evidence were ever tested formally. The same hash can also be submitted to a service such as VirusTotal to check whether the note has been seen in other campaigns, which is often the

quickest way to attribute an incident to a known ransomware family. This value was recorded in the evidence register as item EV-07.

7. Recommendations

Based on what we found during this investigation, our group recommends the following improvements to ABC Electronics, listed roughly in order of the impact they would have had on this specific incident:

- Deploy reputable endpoint antivirus or EDR software on every workstation and server, with real-time protection and tamper protection enabled.
- Implement email filtering with attachment sandboxing so malicious attachments are detonated and blocked before they reach a staff inbox.
- Disable Office macros by default across the organisation, permitting them only from trusted, digitally signed sources.
- Run regular phishing-awareness training and simulated phishing exercises, since a trained user is the control that would have stopped this incident at delivery.
- Apply the 3-2-1 backup rule: three copies of the data, on two different media, with one copy kept offline or offsite, and test restores on a schedule.
- Segment the network with VLANs so a single infected workstation cannot reach every shared drive and server.
- Enforce least privilege so staff accounts have write access only to the shares their role genuinely requires.
- Centrally collect Windows Event Logs through a SIEM or Windows Event Forwarding, and alert on Event ID 1102 so that log clearing cannot go unnoticed.
- Enable process-creation auditing (Event ID 4688) with command-line logging, which would have named the ransomware executable directly.
- Enforce multi-factor authentication on all remote access, email and administrative accounts.
- Patch operating systems, applications and firmware through a formal, documented patch-management schedule.
- Restrict local administrator rights on end-user machines so malware cannot install services or modify system files.
- Write, distribute and rehearse an incident response plan with an escalation contact list, so that the correct first actions — disconnect the network cable, do not power off, notify IT — are automatic.
- Review cyber-insurance cover appropriate to the size and risk profile of the business.

8. Conclusion

This investigation examined a suspected ransomware incident at ABC Electronics in which files were encrypted with a ".locked" extension and a Bitcoin ransom note was left behind after an employee opened a malicious email attachment. Working on the isolated workstation with Windows Event Viewer and PowerShell, our group identified the relevant indicators of compromise, reconstructed a credible attack timeline, and collected and preserved digital evidence in a forensically sound way — an exported .evtx Security log, CSV listings of encrypted files, processes and scheduled tasks, a persistence review of both registry Run keys, and a SHA-256 hash of the ransom note.

Our analysis showed no evidence of remote intrusion, no surviving persistence mechanism and no active command-and-control connection on the host at the time of capture, which supports the conclusion that the infection was delivered locally through the phishing attachment and had completed its encryption routine before the technician isolated the machine. Applying the six-phase incident response lifecycle also let us assess the technician's containment action — physically disconnecting the machine — as the correct first response.

The investigation nevertheless exposed clear gaps in the company's security posture: no formal incident response plan, limited email filtering, a flat network and no staff awareness training. Implementing the recommendations in Section 7 would substantially reduce both the likelihood and the impact of a similar incident. For our group, the project demonstrated in practice how incident identification, evidence collection and preservation, and structured reporting come together in a realistic small-business cybersecurity investigation.

9. References

The following sources were consulted for the incident response methodology, the Windows event identifiers and the PowerShell cmdlets used in this investigation.

1. Cichonski, P., Millar, T., Grance, T. and Scarfone, K. (2012) Computer Security Incident Handling Guide (NIST Special Publication 800-61 Revision 2). National Institute of Standards and Technology. Available at: https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-61r2.pdf (Accessed: 9 August 2026).
2. Microsoft (2025) Audit logon events — Windows Security event IDs 4624 and 4625, Microsoft Learn. Available at: https://learn.microsoft.com/en-us/windows/security/threat-protection/auditing/event-4625 (Accessed: 9 August 2026).
3. Microsoft (2025) Get-WinEvent (Microsoft.PowerShell.Diagnostics) — PowerShell cmdlet reference, Microsoft Learn. Available at: https://learn.microsoft.com/en-us/powershell/module/microsoft.powershell.diagnostics/get-winevent (Accessed: 9 August 2026).

4. Microsoft (2025) Get-FileHash (Microsoft.PowerShell.Utility) — PowerShell cmdlet reference, Microsoft Learn. Available at: https://learn.microsoft.com/en-us/powershell/module/microsoft.powershell.utility/get-filehash (Accessed: 9 August 2026).
5. Microsoft (2025) Get-NetTCPConnection (NetTCPIP) — PowerShell cmdlet reference, Microsoft Learn. Available at: https://learn.microsoft.com/en-us/powershell/module/nettcpip/get-nettcpconnection (Accessed: 9 August 2026).
6. Microsoft (2025) Get-ScheduledTask (ScheduledTasks) — PowerShell cmdlet reference, Microsoft Learn. Available at: https://learn.microsoft.com/en-us/powershell/module/scheduledtasks/get-scheduledtask (Accessed: 9 August 2026).
7. Cybersecurity and Infrastructure Security Agency (CISA) (2023) #StopRansomware Guide. Available at: https://www.cisa.gov/resources-tools/resources/stopransomware-guide (Accessed: 9 August 2026).
8. SANS Institute (2023) Incident Handler's Handbook, SANS Reading Room. Available at: https://www.sans.org/white-papers/33901/ (Accessed: 9 August 2026).
9. Kent, K., Chevalier, S., Grance, T. and Dang, H. (2006) Guide to Integrating Forensic Techniques into Incident Response (NIST Special Publication 800-86). National Institute of Standards and Technology. Available at: https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-86.pdf (Accessed: 9 August 2026).
10. MITRE (2025) Boot or Logon Autostart Execution: Registry Run Keys / Startup Folder (T1547.001), MITRE ATT&CK. Available at: https://attack.mitre.org/techniques/T1547/001/ (Accessed: 9 August 2026).

Appendix A: Quick-Reference Command and Filter Summary

Event Viewer — Event IDs Used

Event ID	Log	Meaning
4624	Security	Successful logon
4625	Security	Failed logon
4720	Security	New user account created
4732	Security	User added to a local security group
4688	Security	New process created
1102	Security	Audit log was cleared
6008	System	Previous system shutdown was unexpected
41	System	Kernel-Power: system rebooted without a clean shutdown
1074	System	Shutdown or restart initiated by a user or process

PowerShell — Commands Used

```
# Create the evidence folder
New-Item -ItemType Directory -Path "C:\Evidence"

# Find encrypted files and export the listing
Get-ChildItem -Path C:\Users -Recurse -Include *.locked -ErrorAction SilentlyContinue
|
  Select-Object FullName, Length, LastWriteTime |
  Export-Csv -Path C:\Evidence\Locked_Files_List.csv -NoTypeInformation

# Top processes by CPU, with executable paths
Get-Process | Sort-Object CPU -Descending |
  Select-Object -First 20 Name, Id, CPU, Path | Format-Table -AutoSize

# Established network connections
Get-NetTCPConnection -State Established |
  Select-Object LocalAddress, LocalPort, RemoteAddress, RemotePort, OwningProcess |
  Sort-Object RemoteAddress

# Persistence: registry Run keys
Get-ItemProperty 'HKCU:\Software\Microsoft\Windows\CurrentVersion\Run'
Get-ItemProperty 'HKLM:\Software\Microsoft\Windows\CurrentVersion\Run'

# Persistence: non-Microsoft scheduled tasks
Get-ScheduledTask | Where-Object {$_.TaskPath -notlike "\Microsoft*"} |
  Select-Object TaskName, TaskPath, State |
  Export-Csv -Path C:\Evidence\Non_Microsoft_Scheduled_Tasks.csv -NoTypeInformation

# Cross-check failed logons
Get-WinEvent -FilterHashtable @{LogName='Security'; Id=4625} -MaxEvents 20 |
  Select-Object TimeCreated, Id, Message | Format-List
```

```
# Evidence integrity
Get-FileHash -Path "C:\Users\tlotl\Documents\READ_ME_NOW.txt" -Algorithm SHA256
```